Lecture 19: PINNs Applications

Sergei V. Kalinin

Physics-Informed Neural Networks

We have:

- a differential equation $g(x, y) = 0$,
- some data $\{x_j, y_j\}$ and
- a neural network $f(x \mid \theta)$ that approximates y.

For a PINN, we would get a loss function that looks like the following,

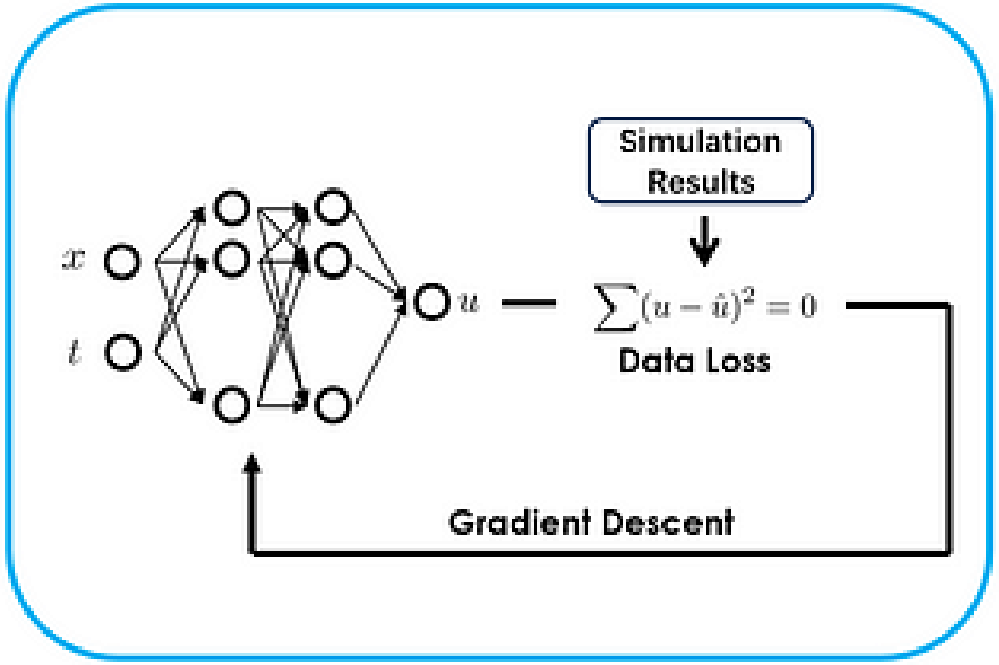
$$Loss_{PINN} = \underbrace{\frac{1}{N}\sum_{j}^{N} ||f(x_j|\theta) - y_j||_2^2}_{\text{Data loss}} + \lambda \underbrace{\frac{1}{M}\sum_{i}^{M} ||g(x_i, f(x_i, |\theta))||_2^2}_{\text{Physics loss}}$$

- Here x_i are *collocation* points. These can be any value we want them to be, usually you would want them to be in the range of values we are interested in.
- The x_j and y_j are our data.
- We can also add a parameter controlling the relative strength of the data loss function and the physics loss function, here we use λ.
- And then just train as you would any other neural network.

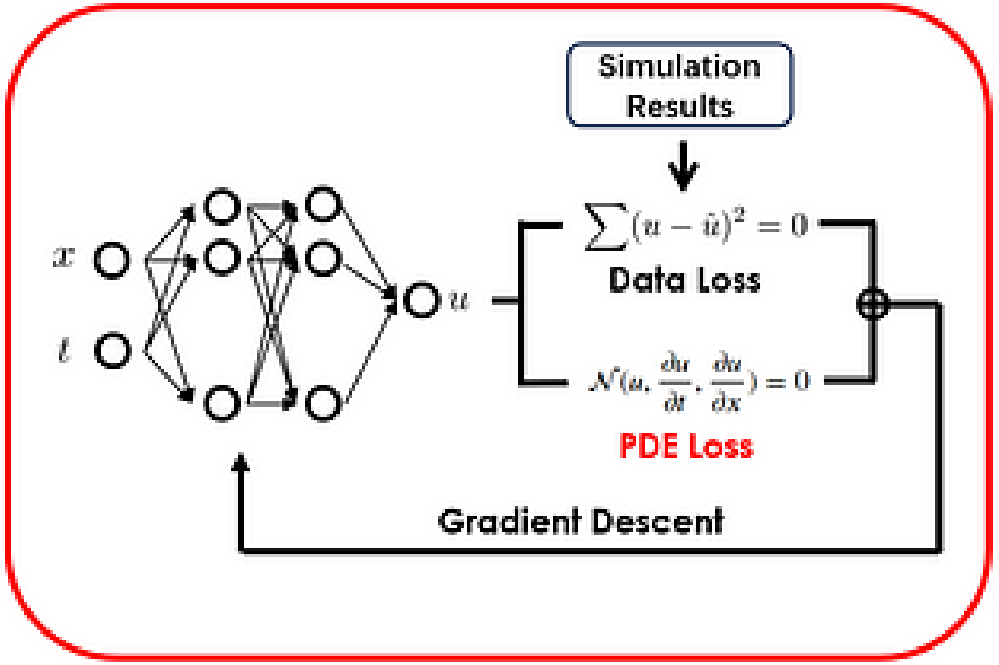
https://medium.com/@theo.wolf/physics-informed-neural-networks-a-simple-tutorial-with-pytorch-f28a890b874a

Colab

Back to functions from data



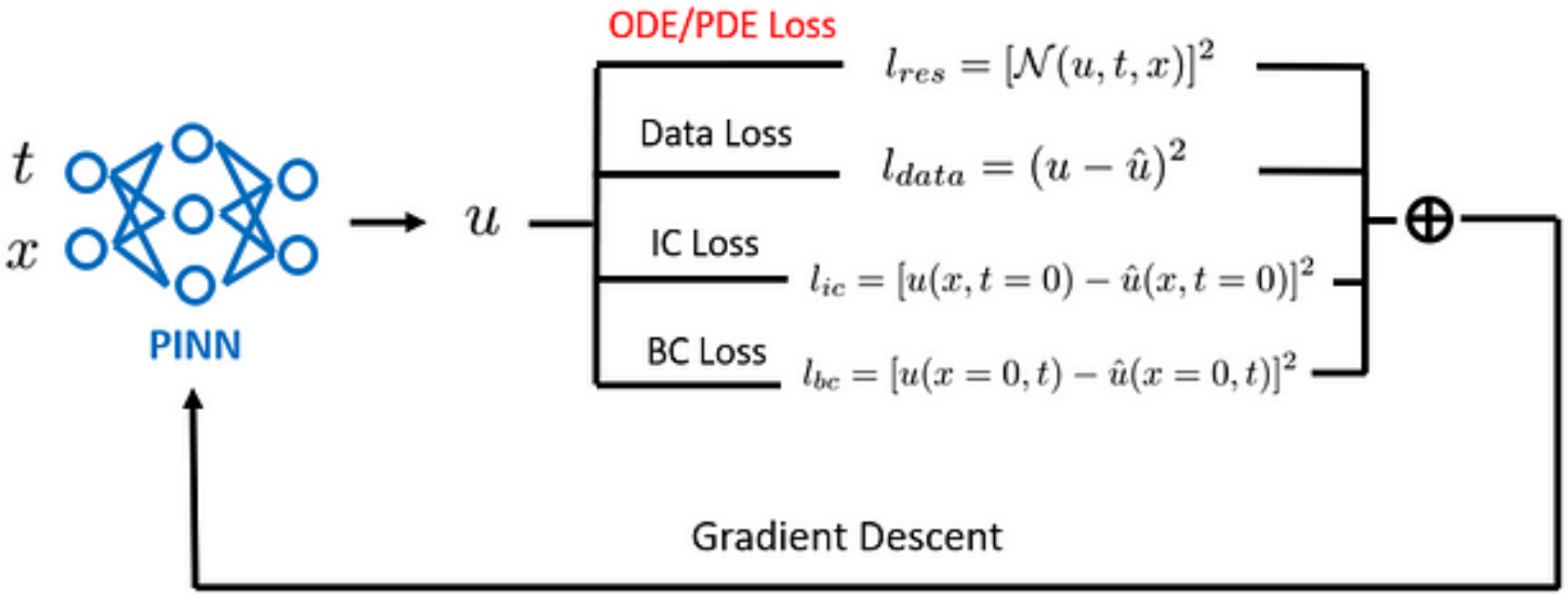
Tranditional NN



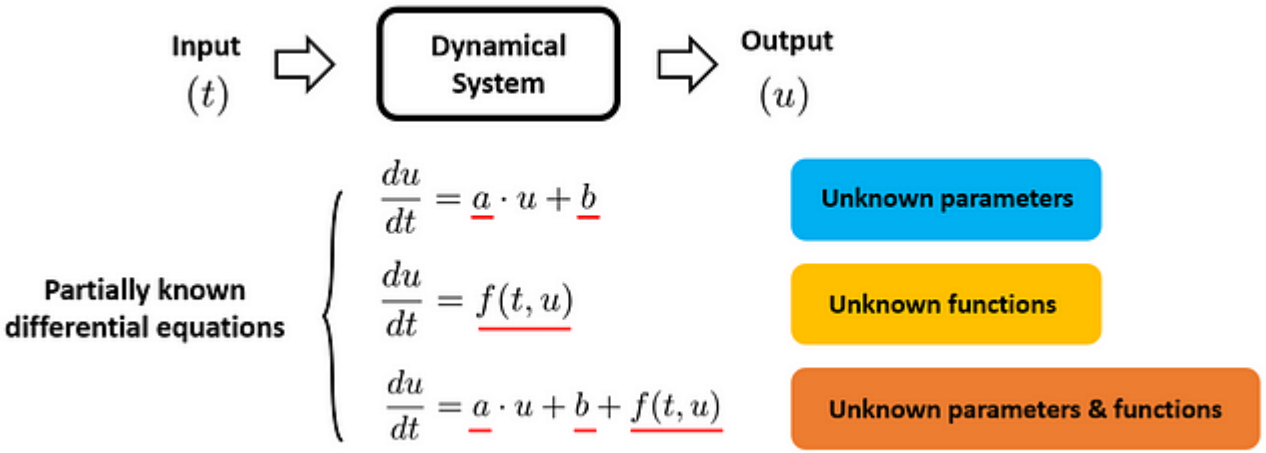
Physics-informed NN

https://towardsdatascience.com/physics-informed-neural-networks-an-application-centric-guide-dc1013526b02

PINN Problems



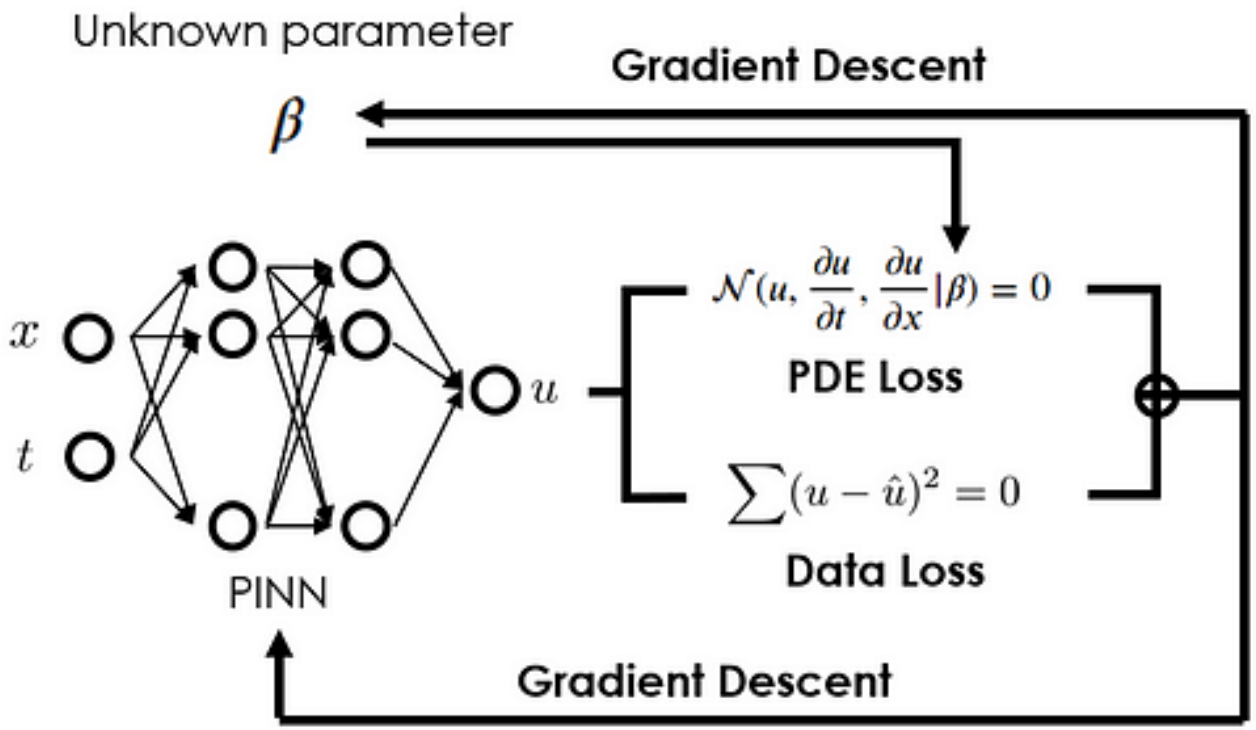
PINN Problems



- The **parameters** of the differential equation are unknown. For example, the governing equations of fluid dynamics are well-established, but the coefficients are highly uncertain.
- The **functional forms** of the differential equations are unknown. For instance, in chemical engineering, the exact functional form of the rate equations may not be fully understood due to the uncertainties in rate-determining steps and reaction pathways.
- Both **functional forms** and **parameters** are unknown. Example is battery state modeling, where the commonly used equivalent circuit model only partially captures the current-voltage relationship (the functional form of the missing physics is therefore unknown). The model itself contains unknown parameters (i.e., resistance and capacitance values).

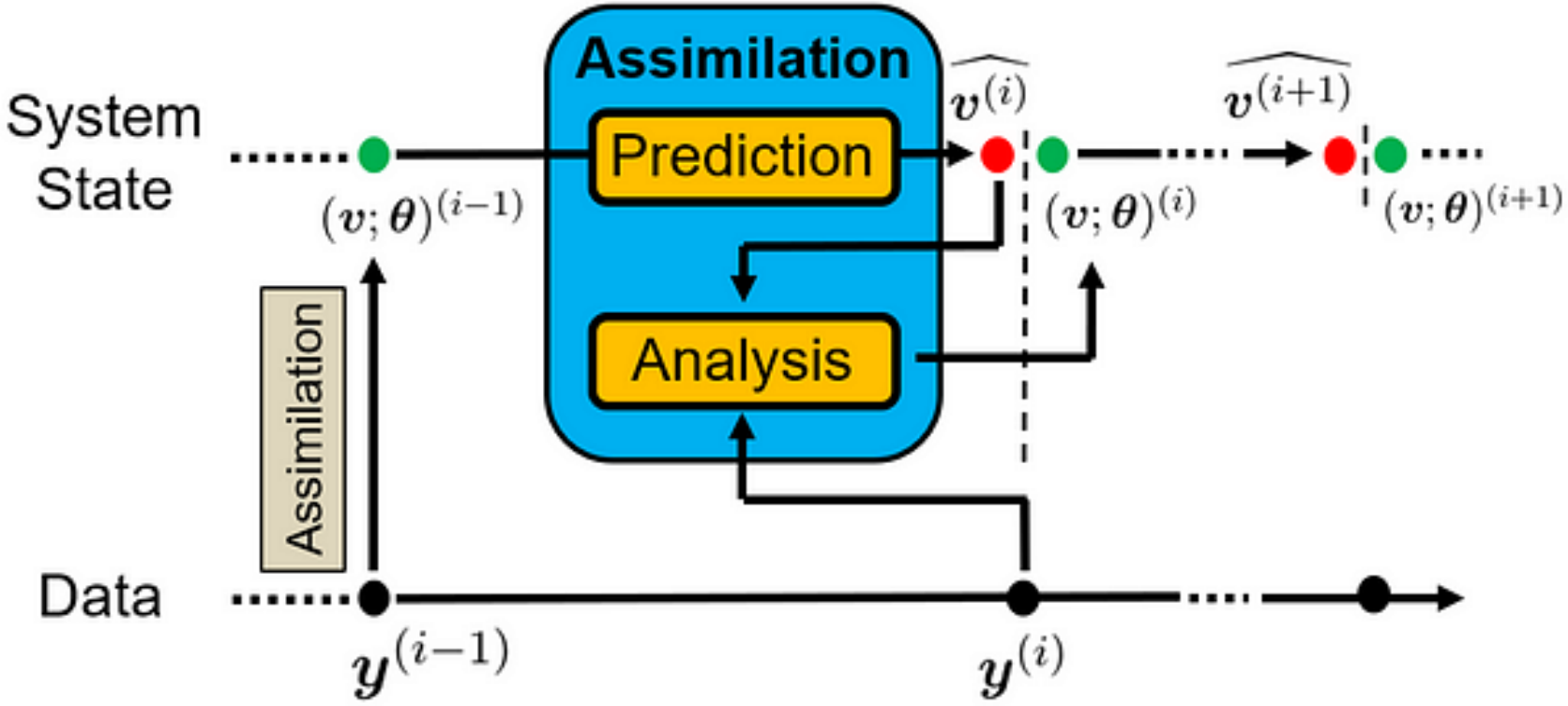
[Discovering Differential Equations with Physics-Informed Neural Networks and Symbolic Regression | by Shuai Guo | Towards Data Science]

Parameter Estimation



https://towardsdatascience.com/physics-informed-neural-networks-an-application-centric-guide-dc1013526b02

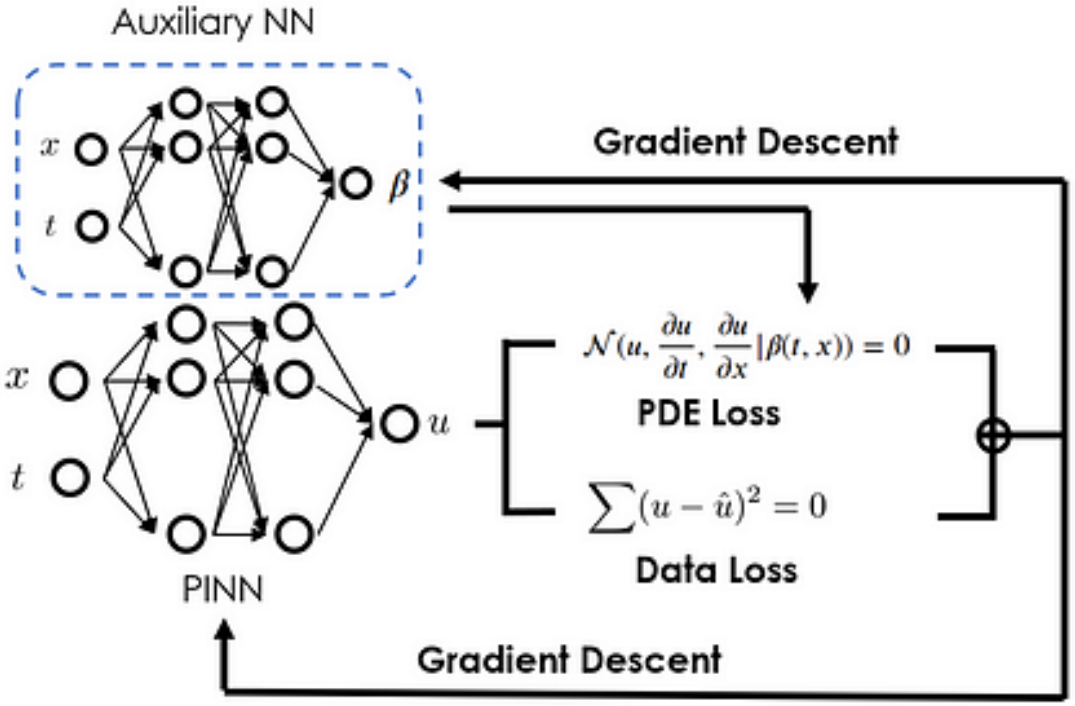
Data Assimilation



https://towardsdatascience.com/physics-informed-neural-networks-an-application-centric-guide-dc1013526b02

System Identification

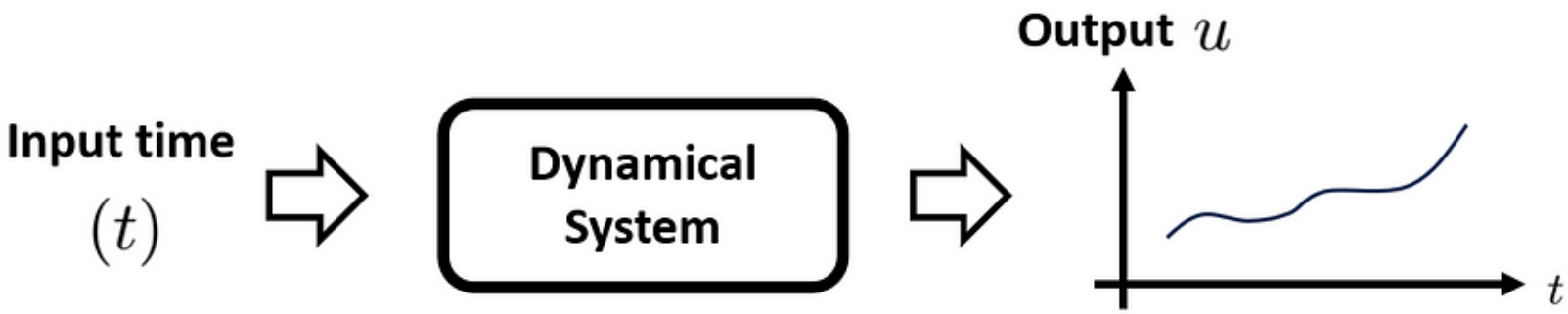
$$\frac{dp}{dt} = f(\cdot) - min(h \cdot p, H_{max})$$



Unlike traditional methods, PINNs are capable of working with partially known differential equations, thus not confined by a complete equation to run simulations.

https://towardsdatascience.com/physics-informed-neural-networks-an-application-centric-guide-dc1013526b02

Neural Differential Equations



Partially known differential equations

$$\begin{cases} \dfrac{du}{dt} = \underline{a} \cdot u + \underline{b} \\ \dfrac{du}{dt} = \underline{f(t,u)} \end{cases}$$

Unknown parameters

Unknown functions

https://medium.com/towards-data-science/modeling-dynamical-systems-with-neural-ode-a-hands-on-guide-71c4cfdb84dc

Neural Differential Equations

$$\frac{du}{dt} = f(u,t) \quad \Rightarrow \quad \frac{du}{dt} \approx f_{NN}(u,t)$$

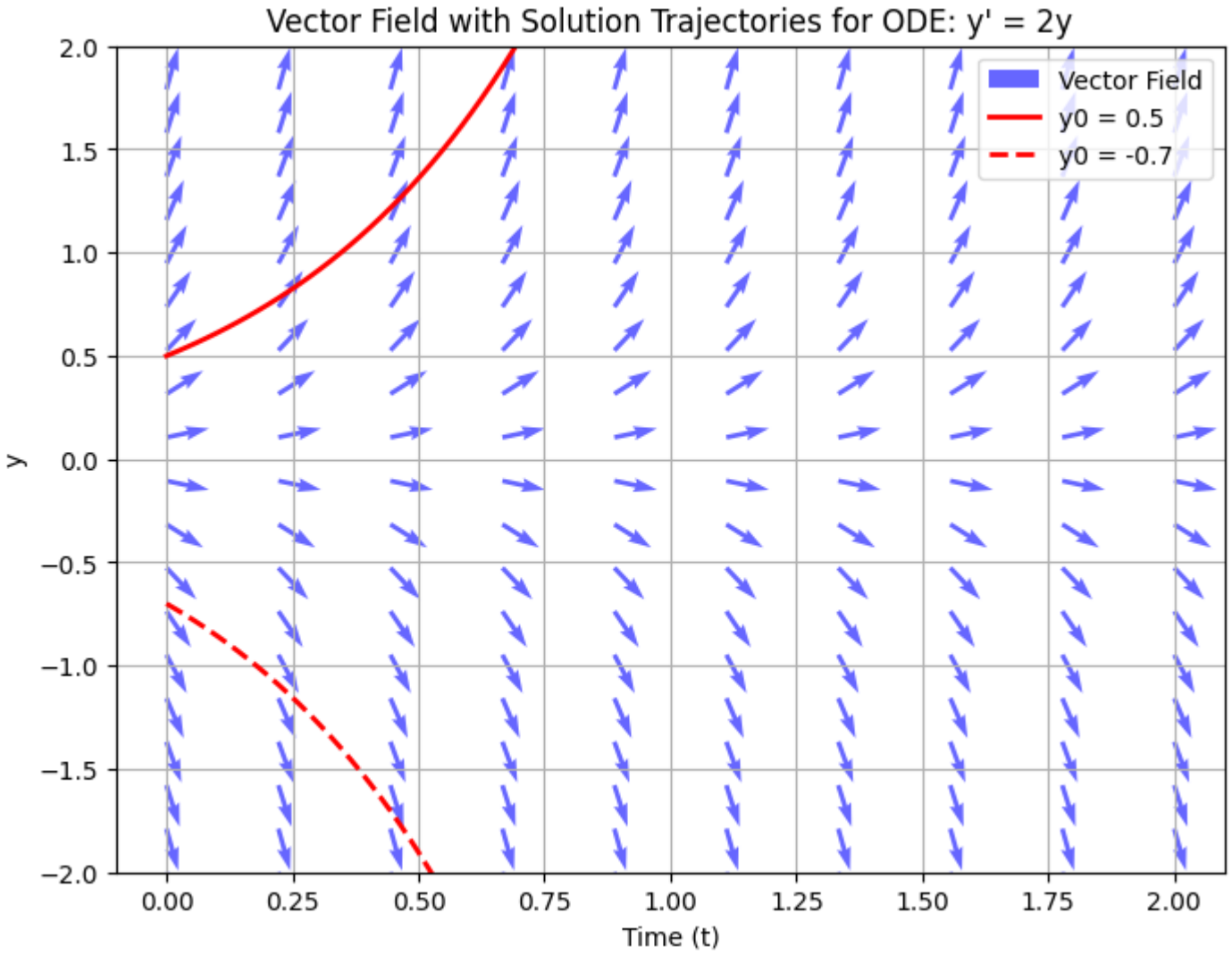
Key idea: Instead of discretizing, use a continuous-depth neural model.

Why is this useful?

- Handles irregular time-series data.
- Provides adaptive computation (solves for arbitrary time points).
- Enables physics-informed learning by integrating known dynamics.

Solving Neural Differential Equations

Forward Pass: Solve ODE using a numerical integrator (RK, Euler, etc.).



Use the forward difference to approximate dx/dt

$$\frac{dx}{dt} \approx \frac{x(t+h) - x(t)}{h}$$

gives the **Euler method** of solving the equation for x(t)

$$x(t+h) = x(t) + h\, f[x(t), t]$$

ResNet and Euler Methods

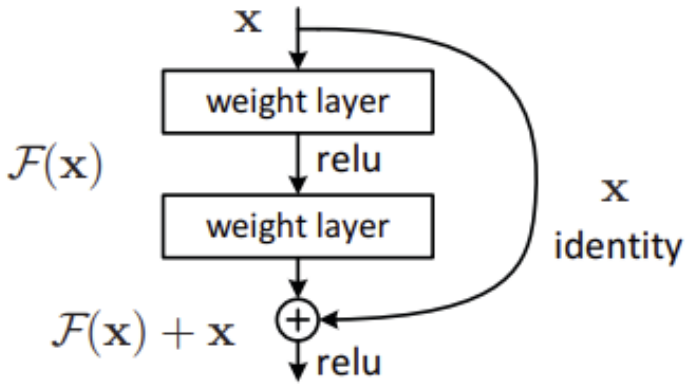


Figure 2. Residual learning: a building block.

$$\mathbf{h}_{t+1} = \mathbf{h}_t + \mathbf{F}(\mathbf{h}_t; \theta_t)$$

arXiv > cs > arXiv:1512.03385

Search...

Help

Computer Science > Computer Vision and Pattern Recognition

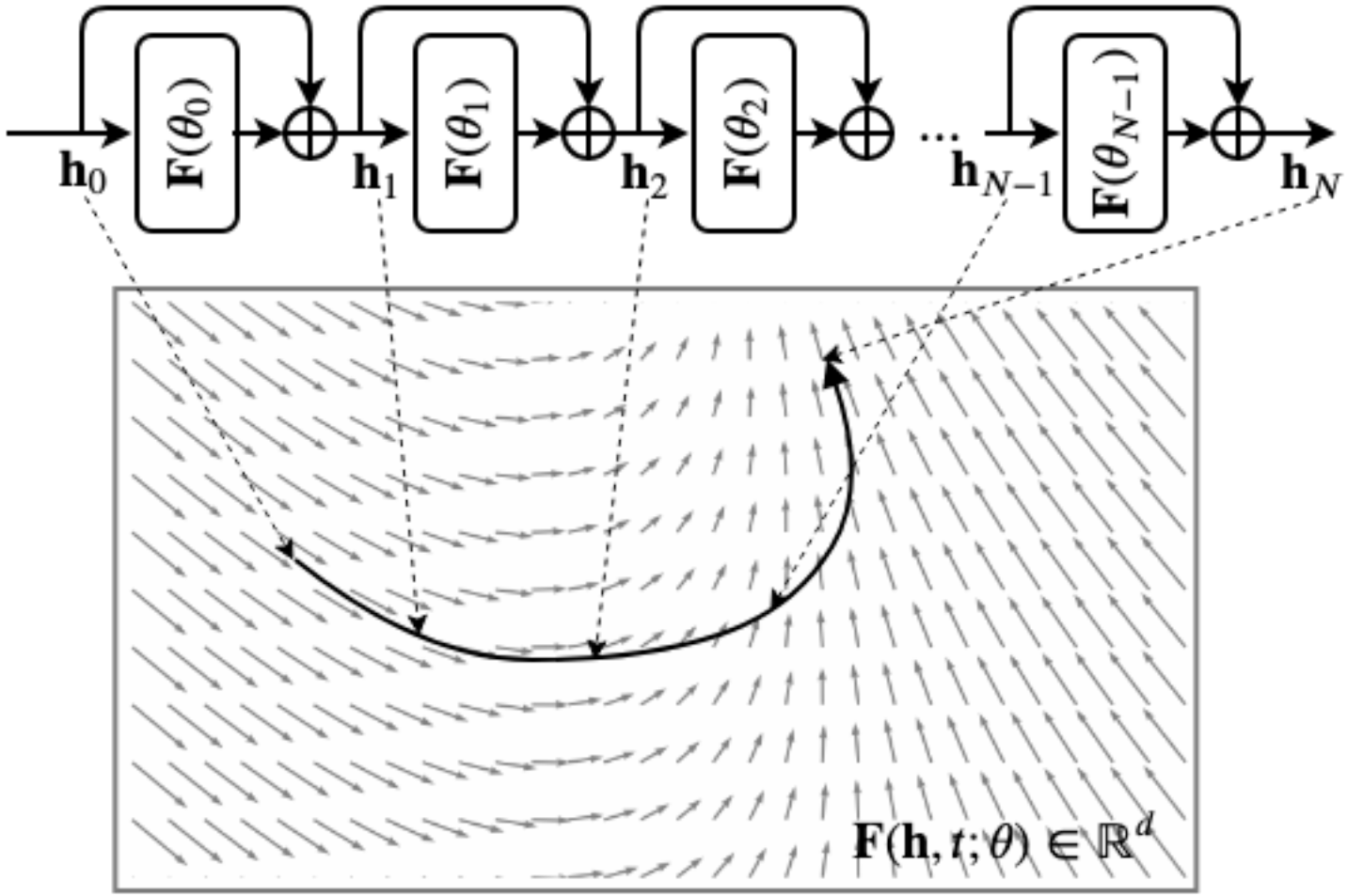
[Submitted on 10 Dec 2015]

Deep Residual Learning for Image Recognition

Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun

Deeper neural networks are more difficult to train. We present a residual learning framework to ease the training of networks that are substantially deeper than those used previously. We explicitly reformulate the layers as learning residual functions with reference to the layer inputs, instead of learning unreferenced functions. We provide comprehensive empirical evidence showing that these residual networks are easier to optimize, and can gain accuracy from considerably increased depth. On the ImageNet dataset we evaluate residual nets with a depth of up to 152 layers---8x deeper than VGG nets but still having lower complexity. An ensemble of these residual nets achieves 3.57% error on the ImageNet test set. This result won the 1st place on the ILSVRC 2015 classification task. We also present analysis on CIFAR-10 with 100 and 1000 layers. The depth of representations is of central importance for many visual recognition tasks. Solely due to our extremely deep representations, we obtain a 28% relative improvement on the COCO object detection dataset. Deep residual nets are foundations of our submissions to ILSVRC & COCO 2015 competitions, where we also won the 1st places on the tasks of ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation.

NeuralODE: Forward Pass



$$\mathbf{h}_N = \text{ODESolve}(start_state = \mathbf{h}_0, dynamics = \mathbf{F}, t_start = 0, t_end = N; \theta)$$

https://ayandas.me/blogs/2020-03-20-neural-ode.html

NeuralODE: Backward Pass

To avoid storing all intermediate states, the adjoint method treats the gradient computation as another ODE and solves it backward in time.

The gradient at the end of the ODE layer is

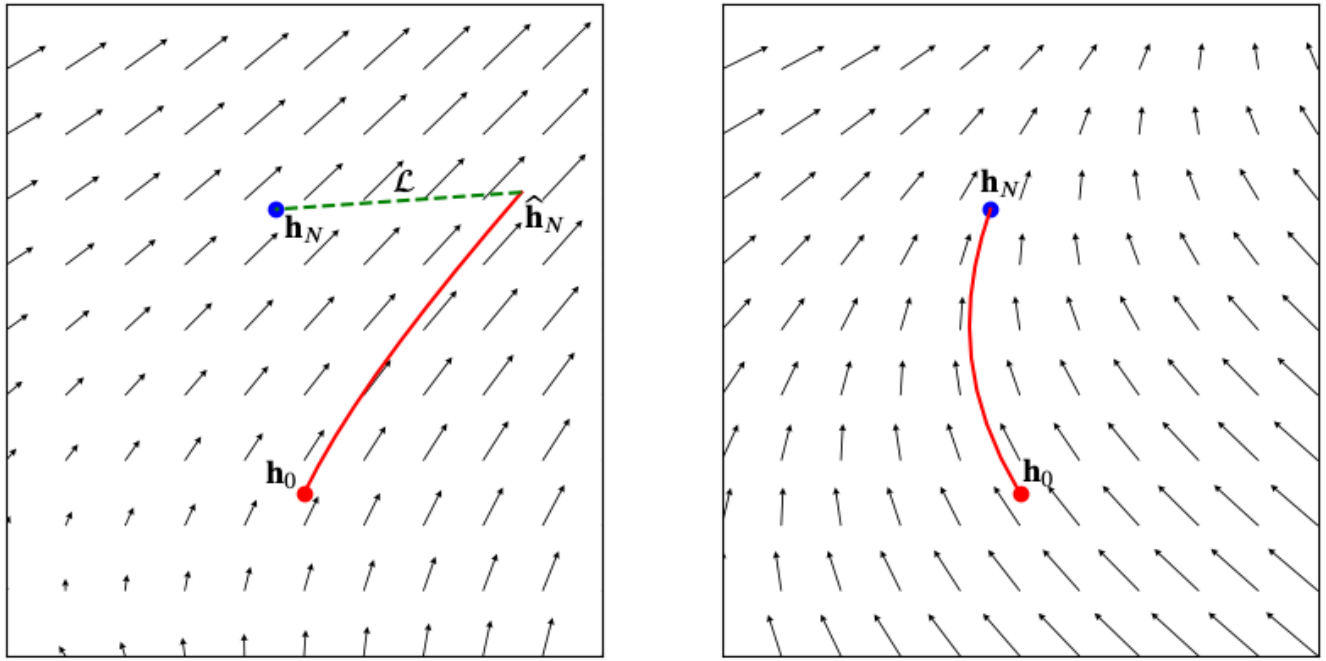
$$\frac{d\tilde{\mathcal{L}}}{d\mathbf{h}_N}$$

where L is a scalar loss.

All we have to do is use this incoming gradient to compute

$$\frac{d\mathcal{L}}{d\theta}$$

and perform an SGD (or any variant) step.



NeuralODE: Backward Pass

Instead of backpropagation, define the adjoint variable $a(t)$ as:

$$\mathbf{a}_t \triangleq \frac{d\mathcal{L}}{d\mathbf{h}_t}$$

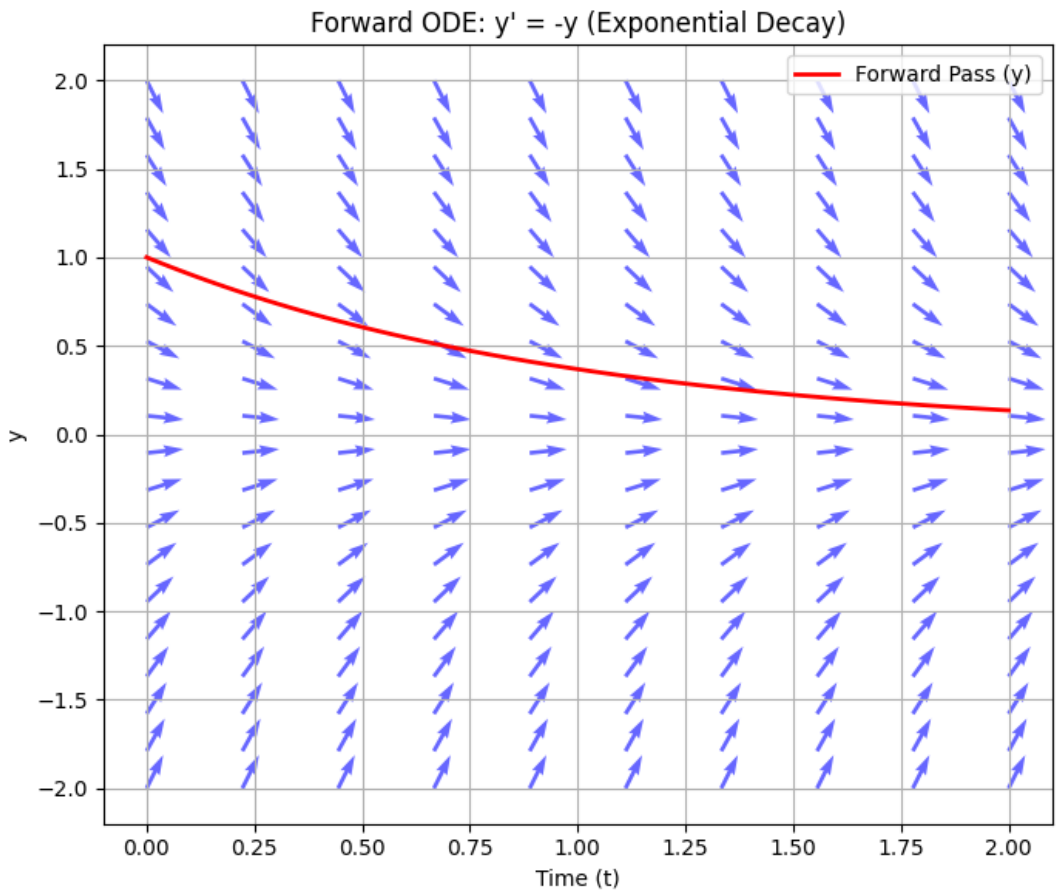
Using the chain rule and integration by parts, we get another ODE for $a(t)$:

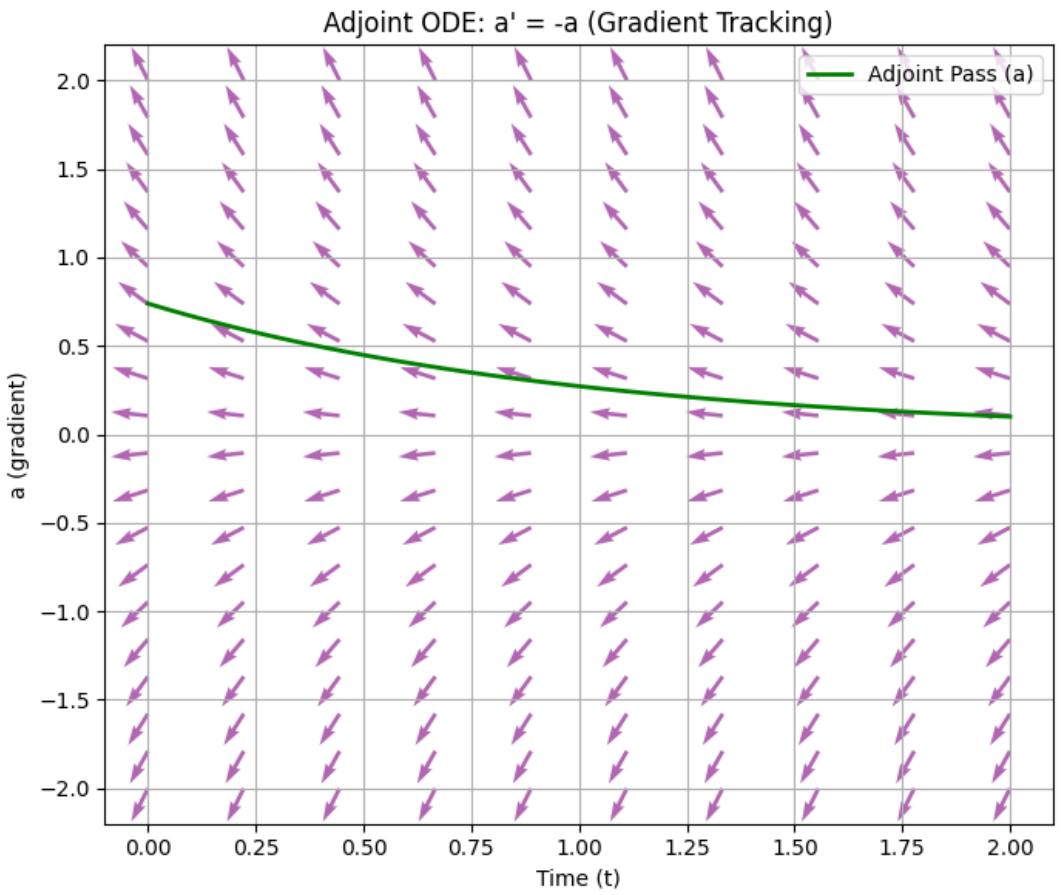
$$\mathbf{F}_a(\mathbf{a}_t, \mathbf{h}_t, t, \theta) \triangleq \frac{d\mathbf{a}_t}{dt} = -\mathbf{a}_t \frac{\partial \mathbf{F}}{\partial \mathbf{h}_t}$$

Thus, we solve this adjoint ODE backward in time from t=T to t=0, instead of storing intermediate states.

$$\mathbf{a}_{N-1}, \cdots, \mathbf{a}_0 = \mathrm{ODESolve}(\mathbf{a}_N, \mathbf{F}_a, N, 0; \theta)$$

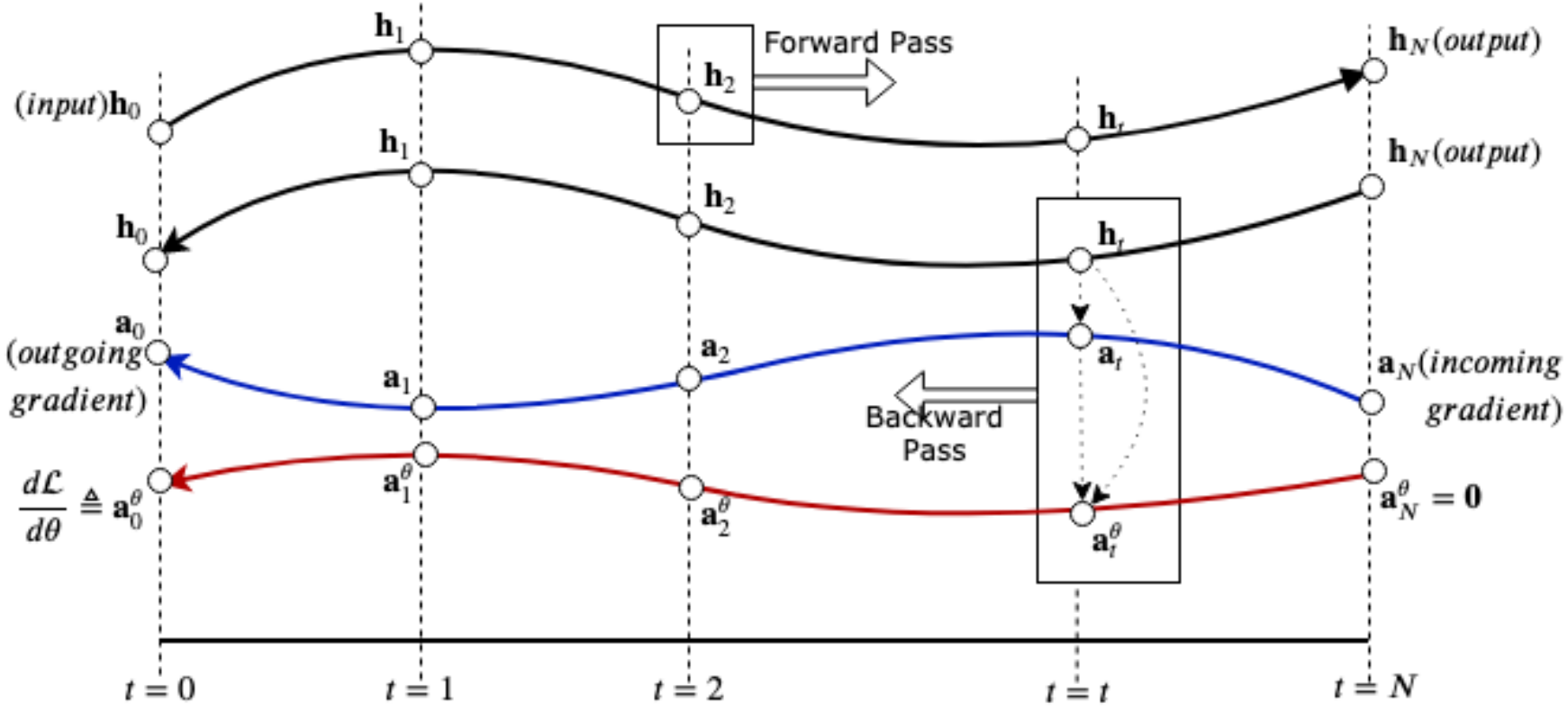
- No need to store intermediate ODE states.
- Works well with adaptive ODE solvers, unlike backpropagation.
- Can be applied to continuous-depth models (e.g., Normalizing Flows, time-series modeling).


Forward ODE: y' = -y (Exponential Decay)
Forward Pass (y)
y
Time (t)
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.0
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00


Adjoint ODE: a' = -a (Gradient Tracking)
Adjoint Pass (a)
a (gradient)
Time (t)
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.0
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00

Solving Neural Differential Equations

$$[\mathbf{a}_{N-1}; \mathbf{h}_{N-1}; _], \cdots, [\mathbf{a}_0; \mathbf{h}_0; \mathbf{a}_0^\theta] = \text{ODESolve}([\mathbf{a}_N; \mathbf{h}_N; \mathbf{0}], [\mathbf{F}_a; \mathbf{F}; \mathbf{F}_\theta], N, 0; \theta)$$



https://ayandas.me/blogs/2020-03-20-neural-ode.html

Learning from Data

$$dx(t)/dt=f\theta(x(t),t)$$

where $f\theta$ is a neural network with trainable parameters θ.
The model learns $f\theta$ such that it **matches observed data points** in time-dependent systems.

1. **Collecting data:** Given time-series data $(x0,x1,...,xT)$; we assume they follow some unknown continuous dynamics.
2. **Forward solving the ODE:**
   - The model starts at **x0** and numerically integrates the learned function $f\theta$.
   - This produces an estimated trajectory $x\hat{}(t)$
3. **Computing the loss:**
   - Compare the predicted trajectory $x\hat{}(t)$ with actual observations $x(t)$
   - A loss function like Mean Squared Error (MSE) is used: $L=\sum i||x\hat{}(ti)-x(ti)||2$
4. **Backpropagating through time:**
   - Instead of backpropagating through an entire computational graph, Neural ODEs use the **adjoint method** to efficiently compute gradients and update parameters θ.

✅ **Handles Noisy & Missing Data** → It can learn smooth representations despite measurement gaps.
✅ **Adaptive Time Steps** → Neural ODEs can predict at any time point, even between observed data points.
✅ **Generalization** → Once trained, they can **predict system behavior beyond observed time points**.

Learning from Data

1 **Physics-Informed Loss Functions** – Penalize deviations from known equations in the loss function.
2 **Hybrid Models (Neural ODE + Known Equations)** – Use partial equations and let the Neural ODE learn the missing part.
3 **Encoding Symmetries and Conservation Laws** – Enforce known physical laws (e.g., energy conservation, Hamiltonian dynamics).
4 **Data-Driven Discovery of Unknown Parameters** – Use known equations but let the model learn uncertain parameters.
5 **Regularization for Smoothness and Stability** – Add constraints to avoid physically unrealistic behavior.
6 **Using Known Boundary and Initial Conditions** – Constrain Neural ODE predictions based on observed conditions.
7 **Learning in Latent Space** – Encode domain knowledge in a latent representation before solving the Neural ODE.

Population Dynamics

$$\frac{dp(t)}{dt} = rp(t)(1 - \frac{p(t)}{k}) - H(p(t))$$

$p(t)$ is the **population size** at time t,
r is the intrinsic **growth rate** of the population,
k is the **carrying capacity** of the environment.

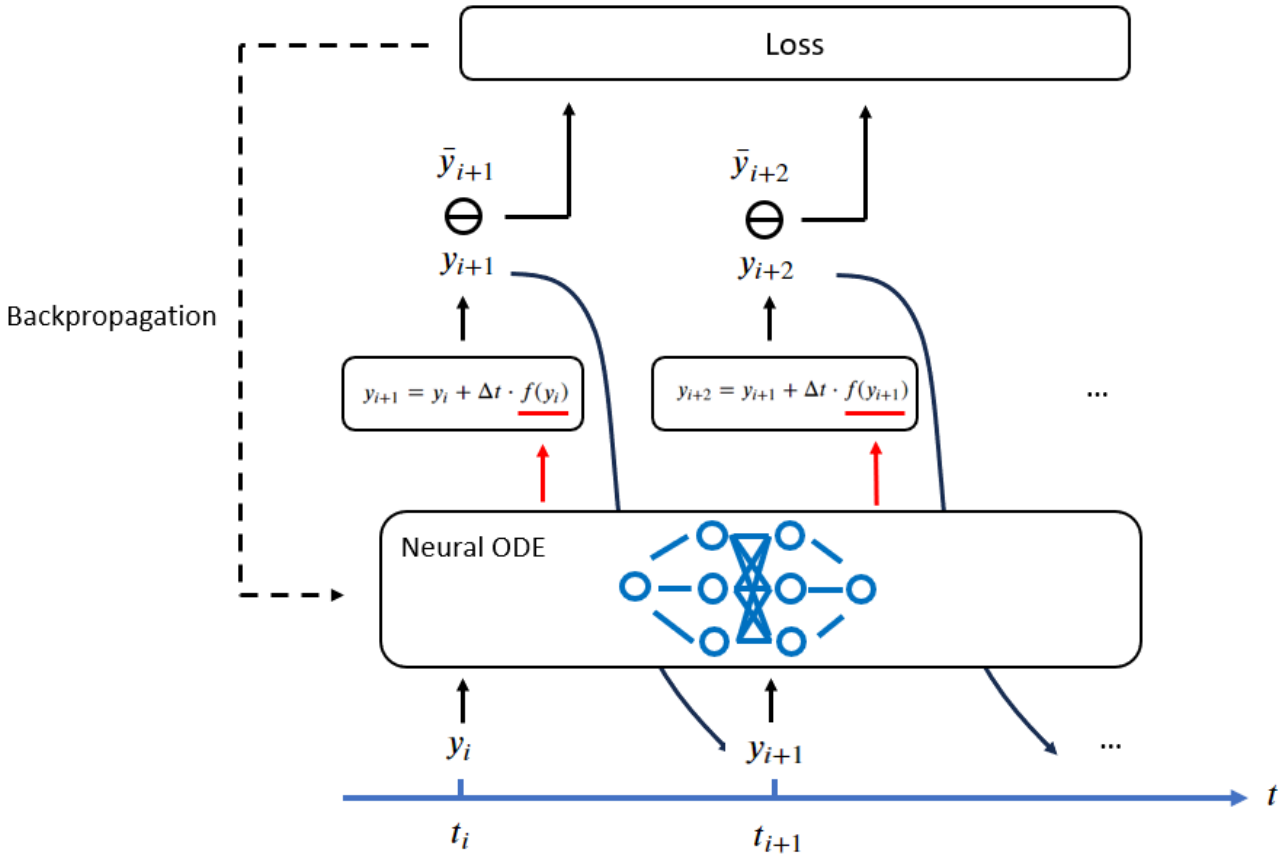
$$H(p(t)) = min(h \cdot p, H_{max})$$

$H(\cdot)$ is the harvesting function that reflects the removal of the animal population due to harvesting (or culling). Here h denotes the harvesting rate and Hmax represents the maximum harvesting capacity.

https://medium.com/towards-data-science/modeling-dynamical-systems-with-neural-ode-a-hands-on-guide-71c4cfdb84dc

Neural Differential Equations

$$\frac{dp}{dt} = f_{unknown}(p) - min(h \cdot p, H_{max})$$



https://medium.com/towards-data-science/modeling-dynamical-systems-with-neural-ode-a-hands-on-guide-71c4cfdb84dc

Population Numerical vs. Neural ODEs

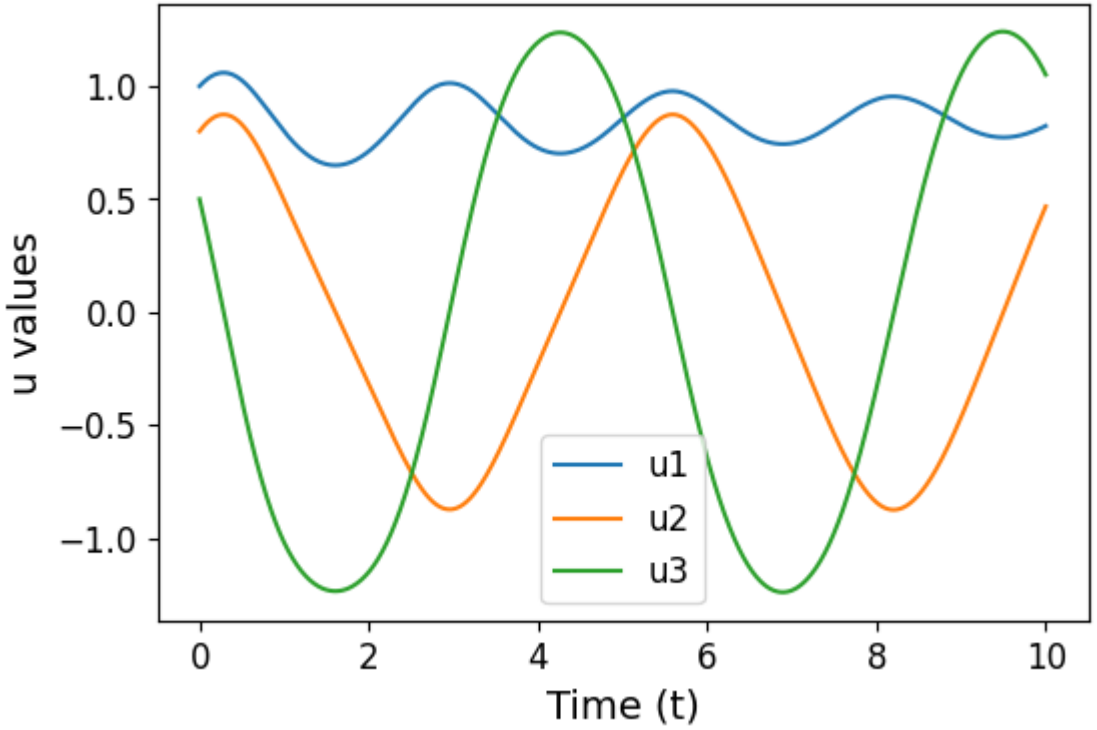
Feature	Numerical ODE	Neural ODE
Solving Known ODEs	✅ Well-suited for explicit equations	✅ Can approximate solutions but requires training
Handling Noisy Data	❌ Struggles with noise and missing data	✅ Learns from noisy data and handles gaps well
Data-Driven Discovery	❌ Requires explicit formulation	✅ Learns hidden dynamics directly from data
Adaptive Computation	✅ Adaptive solvers exist but can be computationally expensive	✅ Naturally adaptive and efficient with adjoint methods
Model Predictive Control (MPC)	✅ Used in control but limited to predefined models	✅ Integrates well into MPC workflows
High-Dimensional Systems	❌ Struggles with large-scale systems	✅ Scales efficiently to high-dimensional problems
Generalization Beyond Training	❌ Cannot extrapolate well beyond observed scenarios	✅ Generalizes better if trained properly
Computational Efficiency	✅ Fast for small problems but costly for complex ones	❌ Training is expensive but efficient at inference

Applications of NeuralODE

- Scientific Computing & Physics-Based Modeling → Solving differential equations, fluid dynamics, reaction kinetics.
- Time-Series Analysis → Financial forecasting, healthcare, climate modeling, power grid optimization.
- Machine Learning & AI Architectures → Normalizing flows, sequence modeling, memory-efficient deep learning.
- Control & Robotics → Optimal control, reinforcement learning, motion planning for autonomous systems.
- Biomedical & Healthcare Applications → Neural activity modeling, drug response prediction, growth modeling.
- Anomaly Detection & Industrial Systems → Fault detection in engineering, finance, and cybersecurity.

Craichnan-Orzag Equations

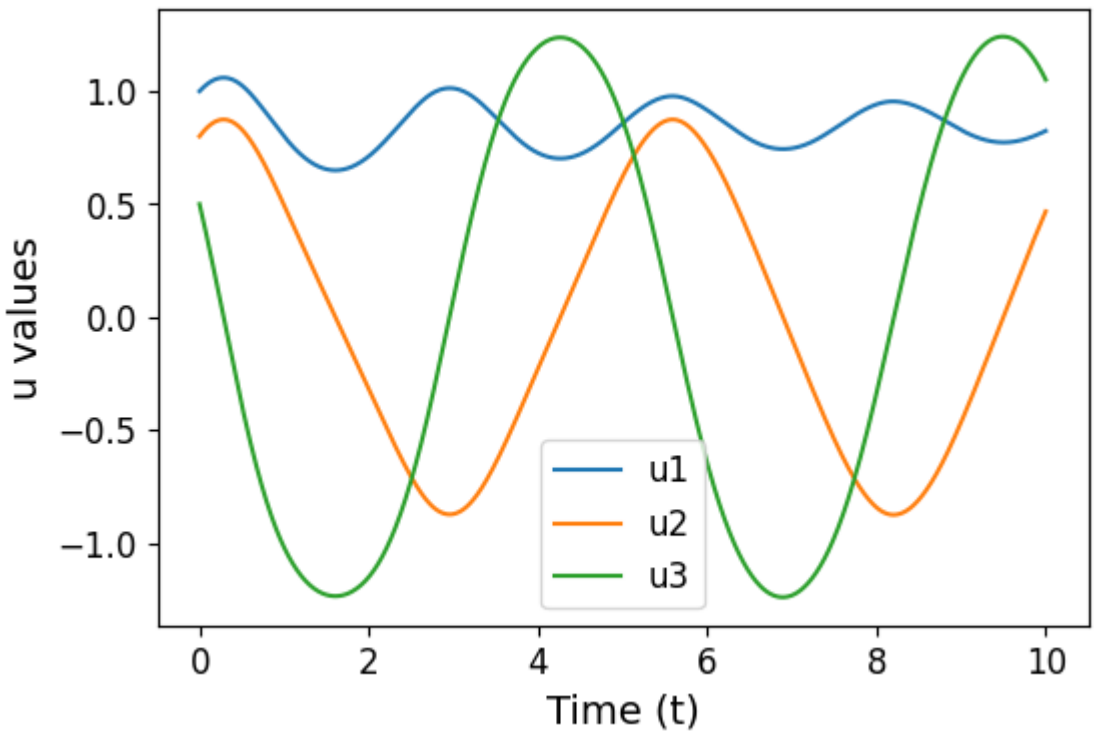
$$\frac{du_1}{dt} = e^{-t/10} u_2 u_3$$
$$\frac{du_2}{dt} = u_1 u_3$$
$$\frac{du_3}{dt} = -2 u_1 u_2$$



$u_1(0)=1$, $u_2(0)=0.8$, $u_3(0)=0.5$

https://medium.com/towards-data-science/discovering-differential-equations-with-physics-informed-neural-networks-and-symbolic-regression-c28d279c0b4d

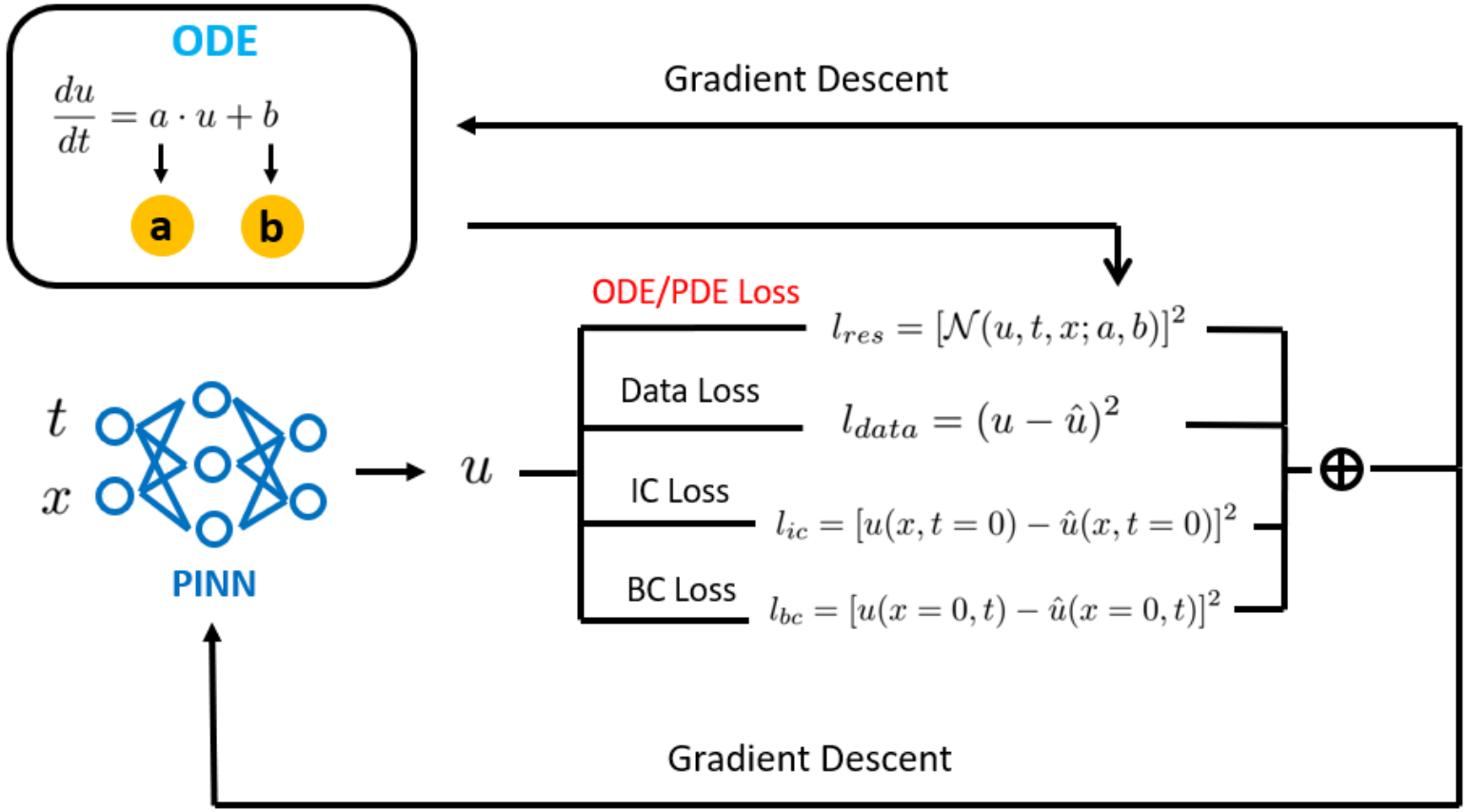
System Identification



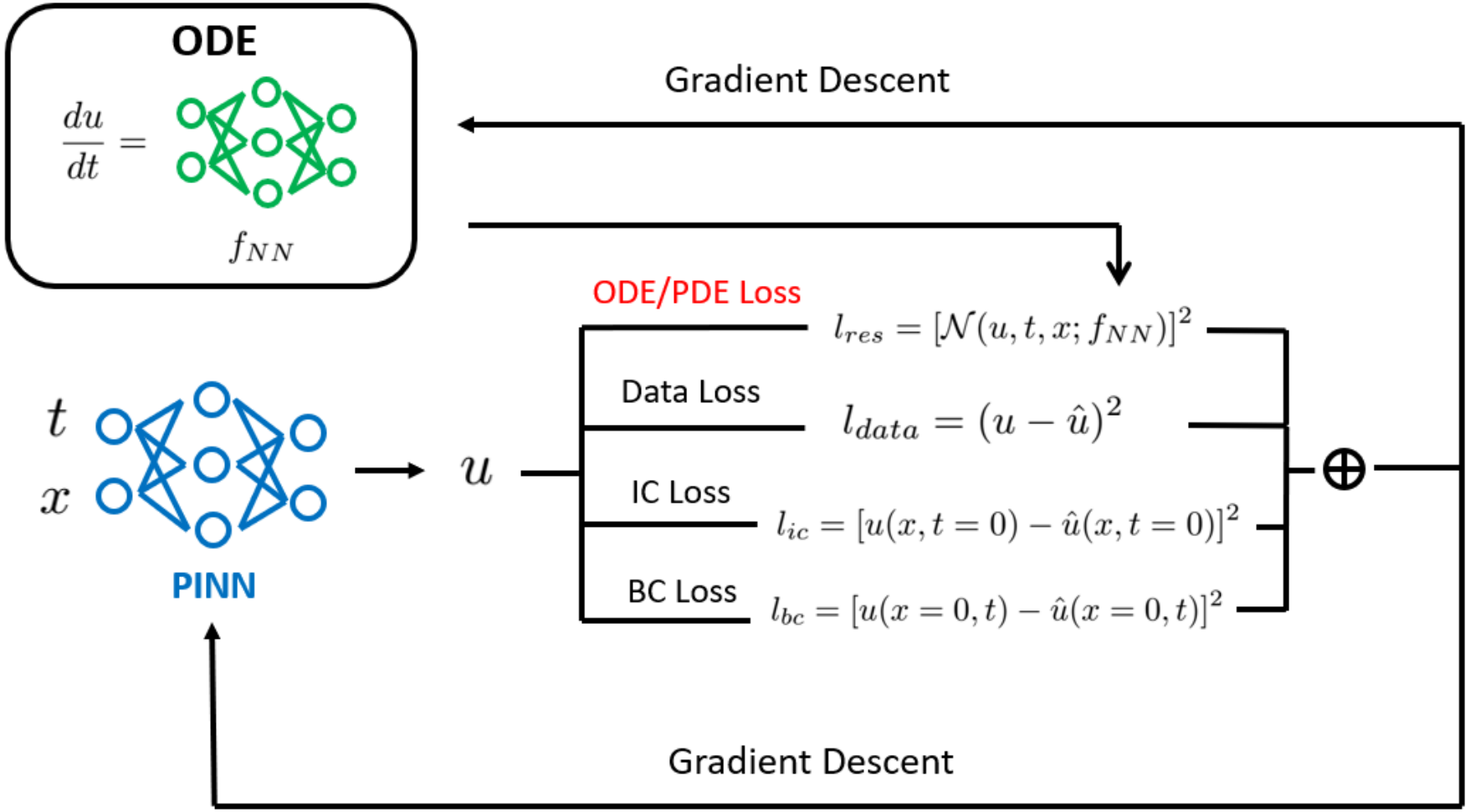
$$\frac{du_1}{dt} = f_1(t, u_1, u_2, u_3)$$
$$\frac{du_2}{dt} = f_2(t, u_1, u_2, u_3)$$
$$\frac{du_3}{dt} = au_1u_2 + b$$

Here, f_1 and f_2 denote the unknown functions, and a and b are the unknown parameters. **Our objective is to calibrate the values of a and b, as well as estimate the analytical functional form of f_1 and f_2.**

https://medium.com/towards-data-science/discovering-differential-equations-with-physics-informed-neural-networks-and-symbolic-regression-c28d279c0b4d

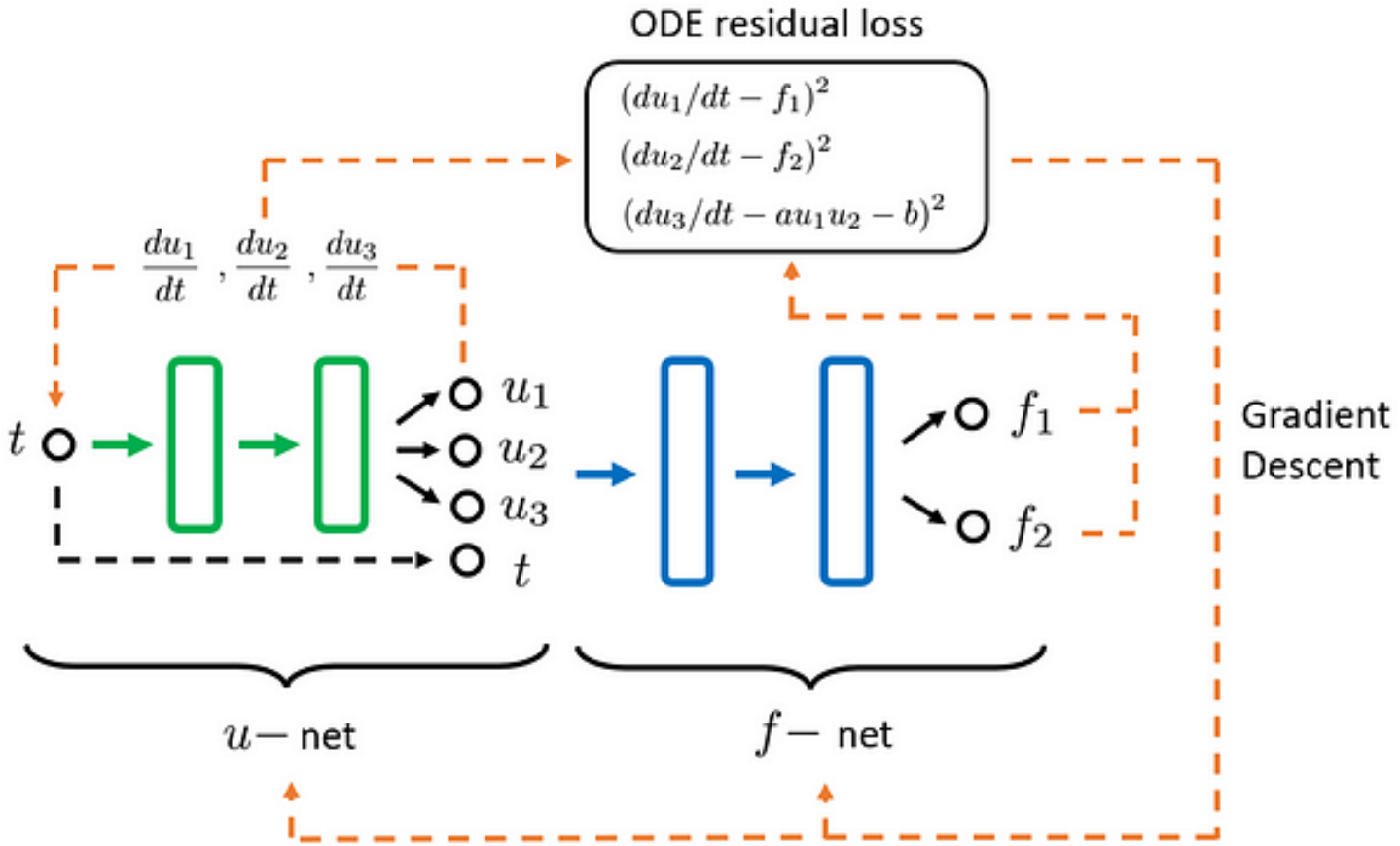


https://medium.com/towards-data-science/discovering-differential-equations-with-physics-informed-neural-networks-and-symbolic-regression-c28d279c0b4d

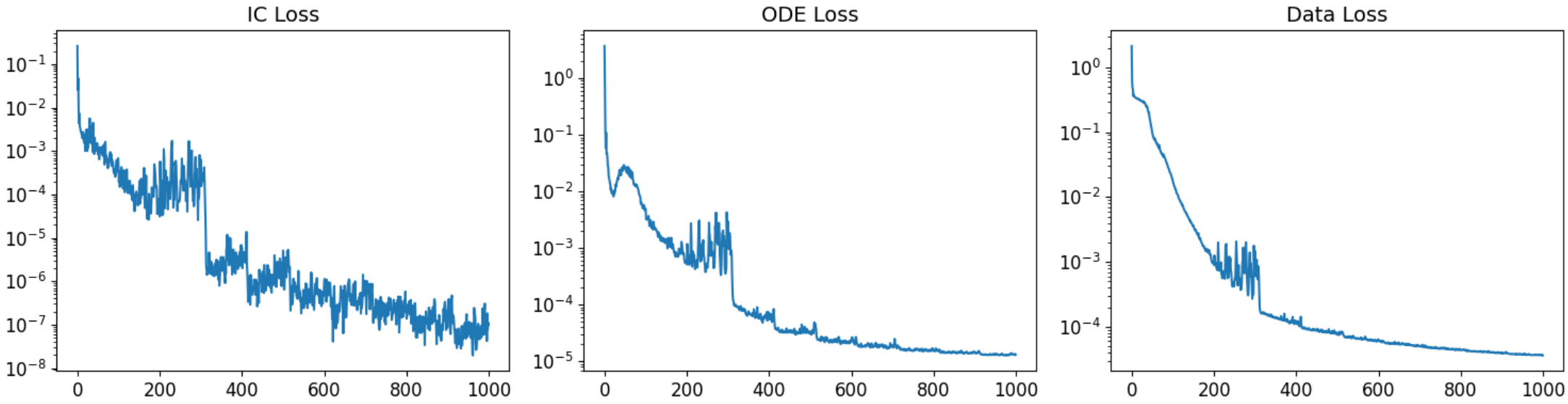


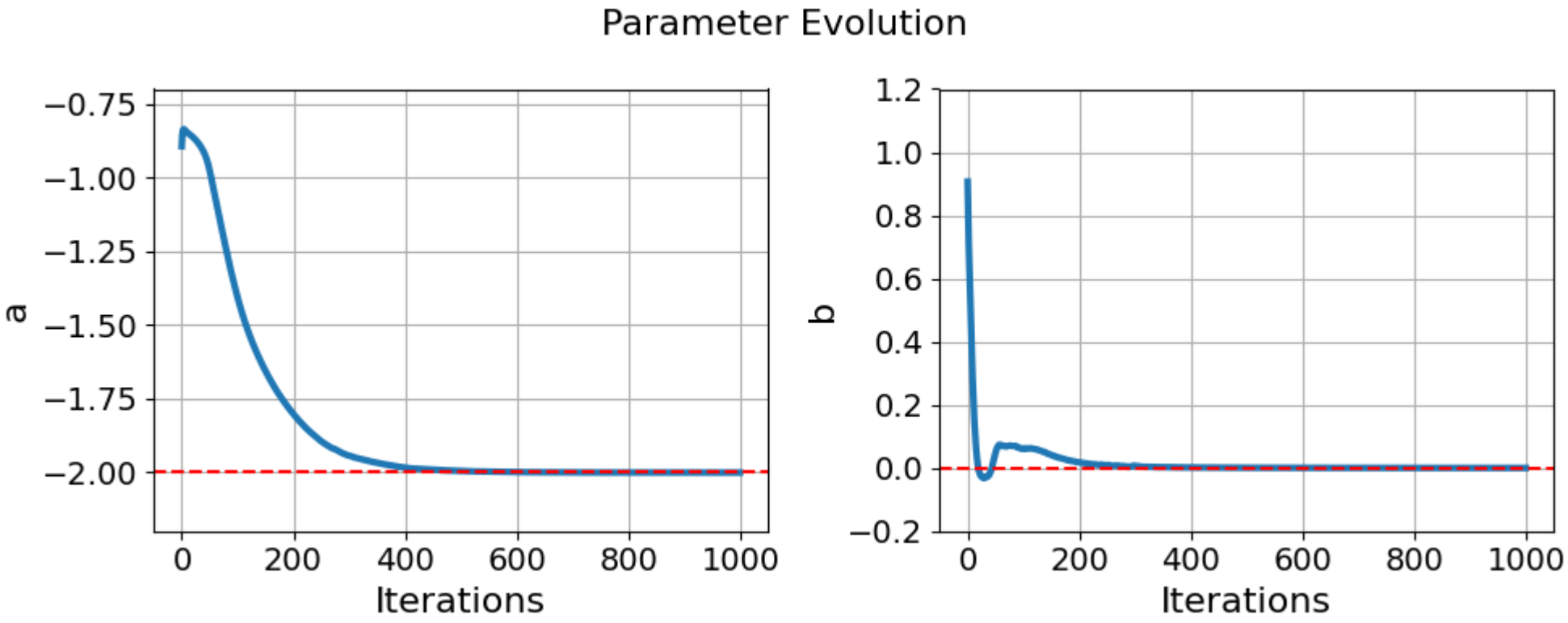
https://medium.com/towards-data-science/discovering-differential-equations-with-physics-informed-neural-networks-and-symbolic-regression-c28d279c0b4d

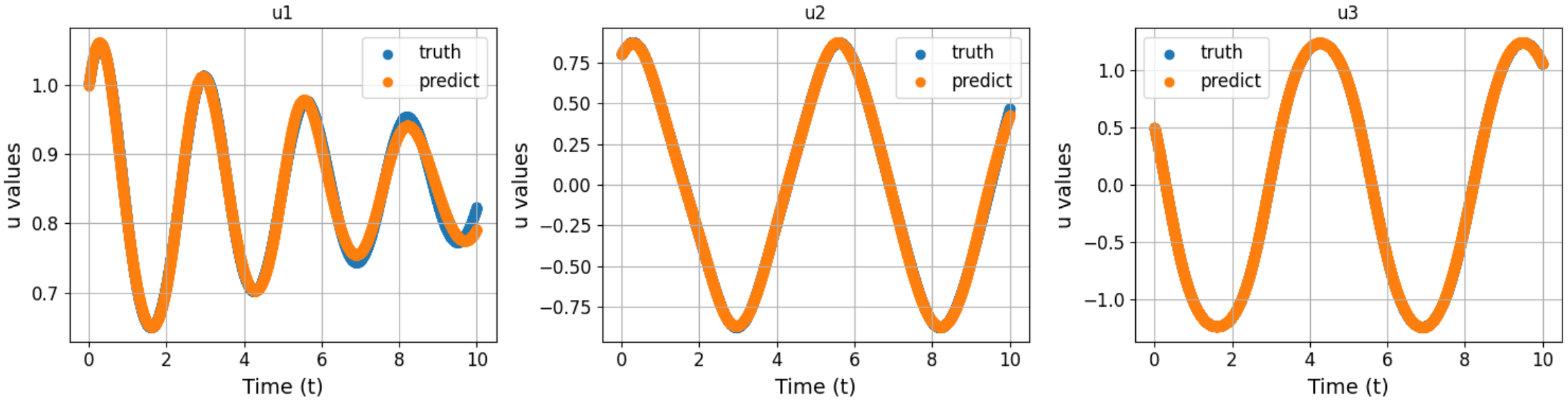
$$\frac{du_1}{dt} = f_1(t, u_1, u_2, u_3)$$
$$\frac{du_2}{dt} = f_2(t, u_1, u_2, u_3)$$
$$\frac{du_3}{dt} = au_1u_2 + b$$

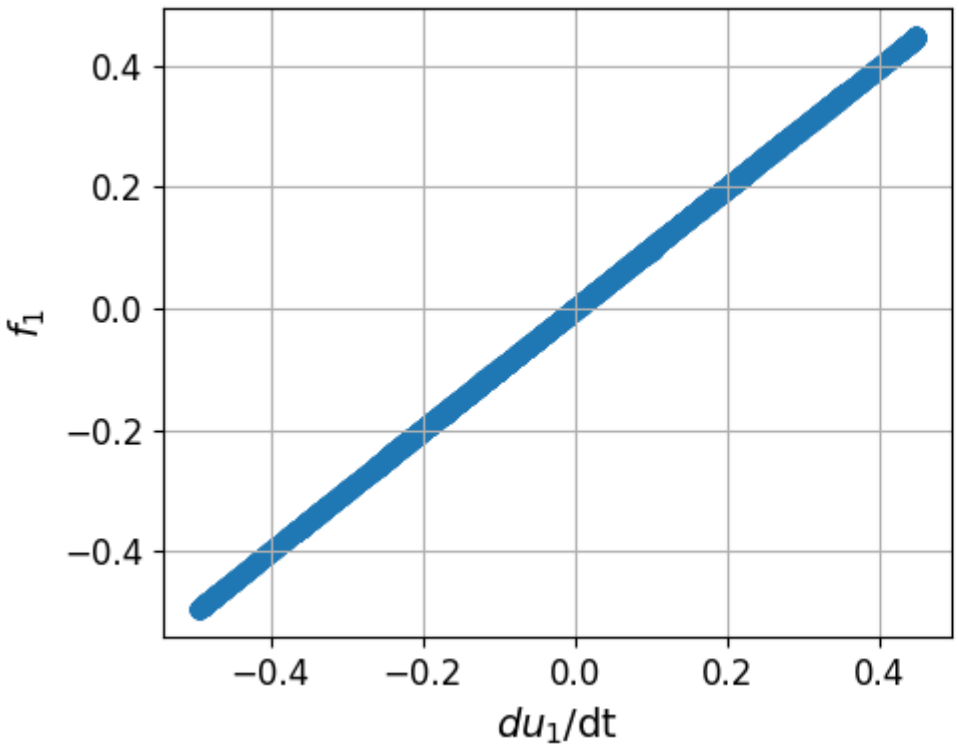


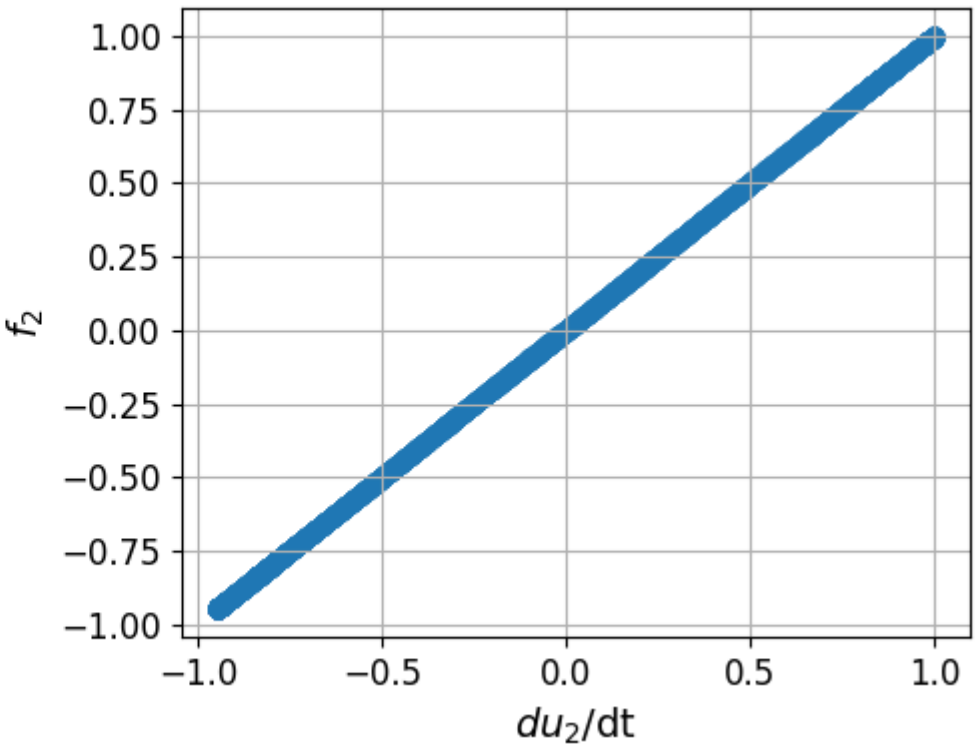
https://medium.com/towards-data-science/discovering-differential-equations-with-physics-informed-neural-networks-and-symbolic-regression-c28d279c0b4d


IC Loss
ODE Loss
Data Loss
10^{-1}
10^{-2}
10^{-3}
10^{-4}
10^{-5}
10^{-6}
10^{-7}
10^{-8}
10^{0}
0
200
400
600
800
1000

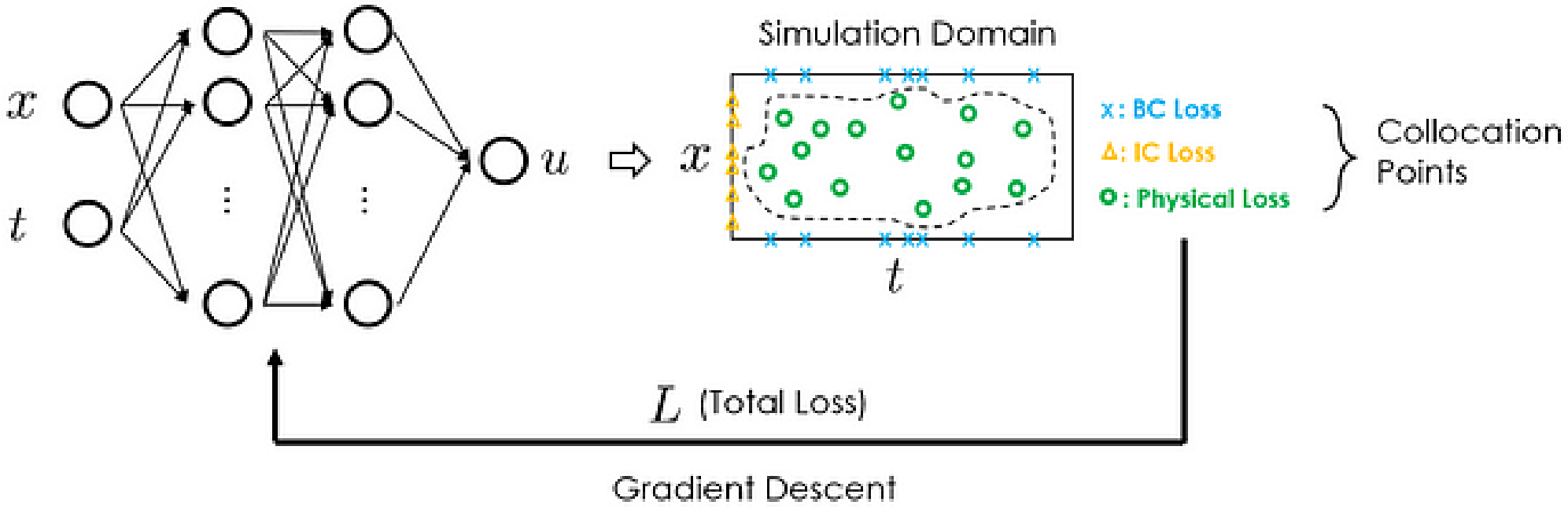

Parameter Evolution
−0.75
−1.00
−1.25
−1.50
−1.75
−2.00
a
0
200
400
600
800
1000
Iterations
1.2
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
b
0
200
400
600
800
1000
Iterations


u1
truth
predict
u values
Time (t)
u2
truth
predict
u values
Time (t)
u3
truth
predict
u values
Time (t)


0.4
0.2
0.0
−0.2
−0.4
f_1
−0.4
−0.2
0.0
0.2
0.4
du_1/dt


1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
f_2
−1.0
−0.5
0.0
0.5
1.0
du_2/dt

PINN Problems



https://towardsdatascience.com/physics-informed-neural-networks-an-application-centric-guide-dc1013526b02

- Compared to the traditional numerical simulation approaches, PINNs are mesh-free. This enables PINNs to easily handle complex simulation domains, reduce manual effort in simulations, and potentially improve computational efficiency.

- Thanks to the universal approximation capability, PINNs are well-suited for modeling complex and nonlinear systems where traditional methods may struggle.

- Compared to the traditional paradigm of supervised learning, PINNs are in principle data-free, i.e., their training does not require collecting input-output pairs, i.e., (t, x, y) -(u, p, T), but entirely based on fulfilling the governing differential equations. However, if scarce data is available, PINNs can also effectively assimilate that.

https://towardsdatascience.com/physics-informed-neural-networks-an-application-centric-guide-dc1013526b02

- Heat equation:

https://inductiva.ai/blog/article/heat-1-an-introduction

https://inductiva.ai/blog/article/heat-2-pinn

https://inductiva.ai/blog/article/heat-3-neurosolver

Helmholz equation

https://towardsdatascience.com/improving-pinns-through-adaptive-loss-balancing-55662759e701

https://towardsdatascience.com/10-useful-hints-and-tricks-for-improving-pinns-1a5dd7b86001

https://towardsdatascience.com/unraveling-the-design-pattern-of-physics-informed-neural-networks-series-01-8190df459527

https://towardsdatascience.com/building-an-expert-gpt-in-physics-informed-neural-networks-with-gpts-75ebf6925966